

Software Architecture and Design Specification

Smart Complaint Management System (SCMS)

Version: 1.0

Authors: Harsh Jha (PES2UG24CS182), Chetan Jevoor (PES2UG24CS132), Harsh Gupta (PES2UG24CS181), Chinnala Manish Ranga (PES2UG24CS135) — Team 9

Date: 15-09-2026

Status: Draft / For Review

Revision History

Version	Date	Author	Change Summary
1.0	15-09-2026	Team 9	First draft of the SAD, written based on our SCMS SRS v1.0

Approvals

Role	Name	Signature / Date
Dev Lead	Harsh Gupta	Pending
QA Lead	Chetan Jevoor	Pending
Product Owner	Chinnala Manish Ranga	Pending
Course Instructor / Evaluator		Pending

1. Introduction

1.1 Purpose

This document describes the software architecture and design of the Smart Complaint Management System (SCMS). It explains how we have split the system into components, why we chose this particular architecture, which technologies we are using, how our REST APIs are structured, two of our main sequence flows, and how we plan to keep the system secure. Everything in this document is based on the requirements written in our SCMS Software Requirements Specification (SRS) v1.0, and this document is meant to guide us while we build, test, and get the project evaluated.

1.2 Scope

This SAD covers the design of the citizen-facing part of SCMS (the complaint lodging form and status tracking), the admin dashboard (used for triage, assignment, and resolution of complaints), the backend REST API and its modules (Auth, Complaint, Tracking, Admin, Notification, Reporting, and Audit Log), the database, and the

notification system. In line with SRS Section 1.2, this document does not cover payment processing, integration with any third-party ticketing system, or a native mobile app, since none of these are part of our project.

1.3 Audience

This document is meant for our own team (developers and testers), our course instructor/evaluator, and anyone else who needs to review, build on, or maintain SCMS.

1.4 Definitions

Term	Definition
JWT	JSON Web Token — a signed token used to keep a user logged in and to carry their role (Citizen/Admin) between the browser and our API.
RBAC	Role-Based Access Control — makes sure a Citizen account cannot open Admin-only pages or APIs.
SLA	Service Level Agreement — the time limit we give ourselves to resolve a complaint before it gets auto-escalated.
ORM	Object-Relational Mapping — a library (SQLAlchemy, Django ORM, or Eloquent) that lets us work with the database using normal code instead of raw SQL.
MVC	Model-View-Controller — the pattern we use to separate data, screens, and the logic that connects them.
CSRF	Cross-Site Request Forgery — an attack where a logged-in user is tricked into sending a request they didn't mean to. We block this with anti-CSRF tokens.
XSS	Cross-Site Scripting — an attack where malicious script is injected into a page. We block this by sanitising input and encoding output.
OTP	One-Time Password — a short code sent to verify a citizen's email or phone number while signing up.
RTM	Requirements Traceability Matrix — maps each requirement ID to a design element, module, and test case (see SRS Section 8).

2. Document Overview

2.1 How to use this document

This document contains our component diagram, a description of every component, the architecture pattern we picked and why, our technology stack, a risk list, a requirement-to-component traceability table, a STRIDE-based threat model, two sequence diagrams for important flows, our REST API design, error-handling/logging notes, UX notes, and a list of things we still need to work on. Wherever possible we refer back to requirement IDs (SCMS-F-####, SCMS-NF-####, SCMS-SR-####) so it is clear which part of the design satisfies which requirement in the SRS.

2.2 Related Documents

- SRS_Smart_Complaint_Management_System_Team_9, v1.0, 05-09-2026 — this is where all the requirement IDs used in this document come from.

- Software Test Plan (STP), v1.0, 15-09-2026 — built from the acceptance test suites described in SRS Section 6.
- Requirements Traceability Matrix (RTM) — included in SRS Section 8 and extended in Section 3.8 of this document.

3. Architecture

3.1 Goals & Constraints

Goals:

- Keep the system secure and reliable, with 99.5% uptime every month (SCMS-NF002).
- Dashboard and search results should come back within 3 seconds for 90% of requests, even with up to 500 people using the system at once (SCMS-NF001).
- The system should be able to scale from 1,000 users to 50,000 users without needing a redesign (SCMS-NF005).
- Code should stay modular enough that a new module can be added in about 3 developer-days (SCMS-NF006).
- Cover the basics of security: TLS everywhere, RBAC, CSRF protection, input sanitisation, and a tamper-evident audit log (SCMS-SR-001 to SR-005).

Constraints:

- We have to build this using Python (Flask/Django) or PHP (Laravel), since that is what was specified for the project (SRS 2.1, 2.5).
- We must use a relational database (PostgreSQL/MySQL) accessed through an ORM.
- The app has to run on normal, low-cost hosting — nothing proprietary or expensive, since this is a student project (SRS 2.5).
- All code has to live on GitHub, and all tasks/requirements are tracked on Jira.

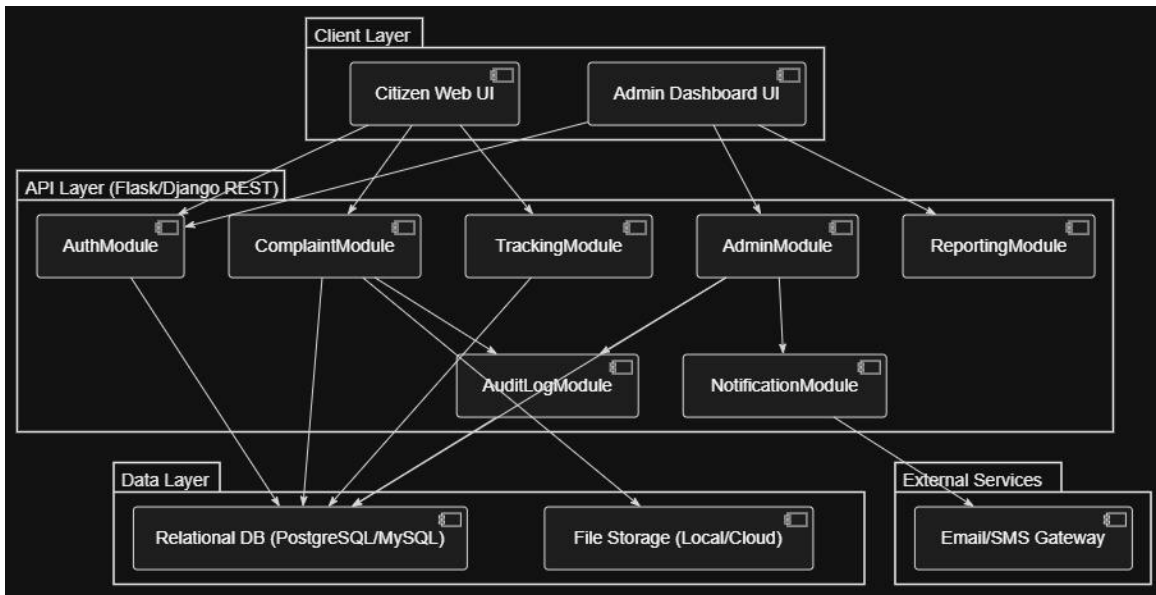
3.2 Stakeholders & Concerns

Stakeholder	What they care about
Complainant (Citizen)	An easy-to-use complaint form, being able to see their complaint status quickly, their personal data and photos staying private, and the site working well on a phone (360px and up).
Admin / Officer	A dashboard that's easy to search/filter/sort, knowing when a complaint is about to breach its SLA, a quick assignment and resolution flow, and notifications actually reaching citizens.
Regulators / Security Reviewers	PII encrypted at rest and in transit, TLS 1.2+ enforced everywhere, Admin routes locked away from Citizen accounts, and an audit trail that can't be silently edited.
Evaluators / Course Instructors	Whether our code is modular and follows MVC, whether the RTM is complete, whether test cases cover the acceptance criteria, and whether our architecture decisions are actually justified.

Stakeholder	What they care about
Developers (our own team)	Clear and maintainable API contracts, being able to add a new module within about 3 developer-days (NF006), and clear rules for error handling and logging.

3.3 Component (UML) Diagram

The diagram below shows the four layers of SCMS — client, API/application, data, and external services — and how they depend on each other.



3.4 Component Descriptions

- Citizen Web UI — the complaint-lodging form, the status-timeline screen, and the feedback/rating screen (SCMS-F010–F013, F020–F024, F041).
- Admin Dashboard UI — the searchable/filterable complaint list, assignment controls, resolution workflow, and analytics screens (SCMS-F030–F035, F042).
- AuthModule — handles registration with email/OTP verification, login, bcrypt/argon2 password hashing, locking an account after 5 failed attempts, issuing JWTs, and enforcing RBAC (SCMS-F001–F004, SR002).
- ComplaintModule — creates new complaints, generates the unique Complaint ID (format SCMS-YYYY-NNNNN), validates mandatory fields on the server side, and handles photo-evidence uploads (up to 3 images, 5 MB each) (SCMS-F010–F013).
- TrackingModule — looks up complaint status by ID or account, renders the status timeline, and handles reopening a complaint (SCMS-F020–F024).
- AdminModule — handles dashboard queries/filters/sorting, assigning a complaint to an officer/department, updating status with remarks, and SLA-based auto-escalation (SCMS-F030–F035).
- NotificationModule — sends in-app, email, and SMS notifications on status change and SLA escalation, through the external gateway (SCMS-F023, F040).
- ReportingModule — generates category-wise and department-wise resolution/turnaround-time reports (SCMS-F042).

- AuditLogModule — keeps a tamper-evident, timestamped, append-only log of status changes, assignments, and admin actions (SCMS-SR004).
- Relational DB (PostgreSQL/MySQL) — stores accounts, complaints, status history, and audit logs, accessed through an ORM (SQLAlchemy / Django ORM / Eloquent).
- File Storage (local/cloud) — stores the photo evidence uploaded through ComplaintModule.
- Email/SMS Gateway — an external service (SMTP, Twilio, or similar) used for OTP delivery and status notifications.

3.5 Chosen Architecture Pattern and Rationale

We went with a simple Layered (3-Tier) architecture, using MVC (or Django's MVT) inside the application layer: a presentation layer (the Citizen Web UI and Admin Dashboard UI), an application/business-logic layer (our Flask or Django REST modules — Auth, Complaint, Tracking, Admin, Notification, Reporting, AuditLog), and a data layer (the relational database, accessed through an ORM).

We chose this pattern for a few reasons: it fits the technology constraint we were given (Flask/Django or Laravel, SRS 2.1/2.5), it keeps the code modular enough to meet NF006 (adding a new module within 3 developer-days), and it is realistic for a four-person team to actually finish in one semester. We also looked at microservices, but decided against it — at our scale (up to 500 concurrent users per NF001, and 1,000–50,000 total users per NF005), the extra work of service discovery, distributed transactions, and separate deployment pipelines would cost us more than it would help, and it would also go against the constraint of using simple, low-cost hosting (SRS 2.5).

3.6 Technology Stack & Data Stores

- Backend: Python 3.x with Flask or Django REST Framework, split into layered MVC/MVT modules (AuthModule, ComplaintModule, TrackingModule, AdminModule, NotificationModule, ReportingModule, AuditLogModule).
- ORM / data access: SQLAlchemy (Flask) or Django ORM, which gives us parameterised queries and migrations.
- Database: PostgreSQL or MySQL — a relational schema that supports audit logging and traceability, per SRS 2.5.
- Authentication: JWT/session-based tokens (SRS 3.4), with bcrypt or argon2 for password hashing.
- File storage: local disk or cloud object storage, for complaint photo evidence (up to 3 files, 5 MB each).
- Notification gateway: SMTP for email and Twilio (or similar) for SMS/OTP.
- Frontend: a responsive web UI (HTML/CSS/JS or a lightweight JS framework) that works from 360px to 1920px screens (SCMS-NF004).
- Transport & hosting: HTTPS with TLS 1.2+, behind an Nginx reverse proxy, on Linux-based hosting (SRS 2.4).
- Tooling: GitHub for version control, Jira for tracking requirements/tasks, REST/JSON as our API style (SRS 2.5).

3.7 Risks & Mitigations

Risk	Mitigation
Sudden spike in traffic (e.g., a lot of SLA escalations at once)	Scale horizontally behind a load balancer, use connection pooling on the DB, and cache read-heavy dashboard/search queries to stay within NF001 ($\leq 3s$ p90) and NF005 (1k–50k users).
Notification gateway goes down (SMTP/SMS outage)	A retry queue with exponential backoff in NotificationModule, falling back to in-app notification, and alerting when the delivery-failure rate crosses a threshold.
File-upload abuse or storage filling up	Enforce the 3-image / 5 MB limit server-side (SCMS-F013), validate file type, and monitor storage usage.
SQL injection / XSS	Parameterised queries through the ORM, server-side input sanitisation and output encoding, and Content-Security-Policy headers (SCMS-SR003).
Brute-force / credential-stuffing login attempts	Rate-limiting middleware and a 5-attempt account lockout with audit logging (SCMS-F004), plus bcrypt/argon2 password hashing (SCMS-F003).

3.8 Traceability to Requirements

Requirement ID(s)	Component
SCMS-F001–F004 (Registration, Auth, Password hashing, Lockout)	AuthModule
SCMS-F010–F013 (Lodge complaint, ID generation, validation, photo evidence)	ComplaintModule + File Storage
SCMS-F020–F024 (Track status, timeline, notifications, reopen)	TrackingModule
SCMS-F030–F035 (Dashboard, assignment, status update, SLA escalation)	AdminModule
SCMS-F040–F042 (Notifications, feedback, reporting)	NotificationModule / ReportingModule
SCMS-NF001, NF005 (Response time, scalability)	API Layer / Web UI / Infrastructure
SCMS-NF003, SR001 (PII encryption, TLS)	Data Layer / Network
SCMS-SR002 (RBAC)	AuthModule
SCMS-SR003 (Input sanitisation)	API Layer (input validation)
SCMS-SR004 (Tamper-evident audit log)	AuditLogModule
SCMS-SR005 (CSRF protection)	Admin Dashboard UI / API Layer

3.9 Security Architecture

We used the STRIDE model to think through possible threats and how we plan to handle each one.

STRIDE Category	Threat / Requirement	Mitigation
Spoofing	Someone pretends to be a citizen or admin.	JWT-signed session tokens; email/OTP verification at registration (SCMS-F001, F002).
Tampering	Complaint or status records get changed without permission.	Tamper-evident, immutable, timestamped audit log of every status change, assignment, and admin action (SCMS-SR004), through AuditLogModule.
Repudiation	A user denies performing an action that changed something.	Every state-changing action is tied to an authenticated user through JWT claims stored in the audit log (SRS 5.1.1).
Information Disclosure	A citizen's personal data or evidence photos get exposed.	TLS 1.2+ mandatory on all connections (SCMS-SR001); PII encrypted at rest and in transit (SCMS-NF003).
Denial of Service	Brute-force login attempts or request flooding.	Rate-limiting middleware and a 15-minute account lockout after 5 failed login attempts, with audit logging (SCMS-F004).
Elevation of Privilege	A Citizen account tries to reach an Admin/Officer route.	Role-Based Access Control (RBAC) enforced on every route using JWT role claims (SCMS-SR002).

4. Design

4.1 Design Overview

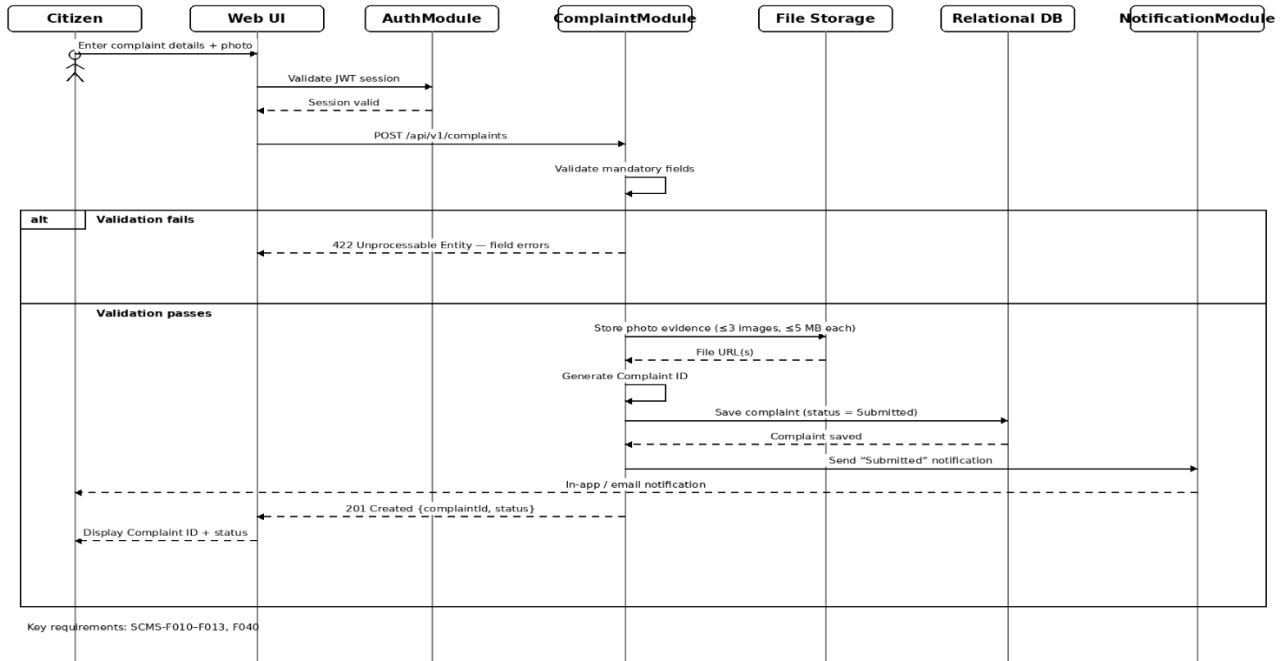
SCMS is built as a set of layered MVC modules sitting behind a REST API. Each business module (Auth, Complaint, Tracking, Admin, Notification, Reporting, AuditLog) has its own controller/service and its own data-access code, and talks to the relational database through the ORM. Keeping things separated like this makes each module easier to test on its own, helps us hit the modular-maintainability target in SCMS-NF006, and keeps the mapping to the RTM in SRS Section 8 clean.

4.2 UML Sequence Diagrams

Below are two of our main flows: (1) a citizen lodging a new complaint with photo evidence, and (2) an admin triaging, assigning, and resolving a complaint, which also triggers a notification.

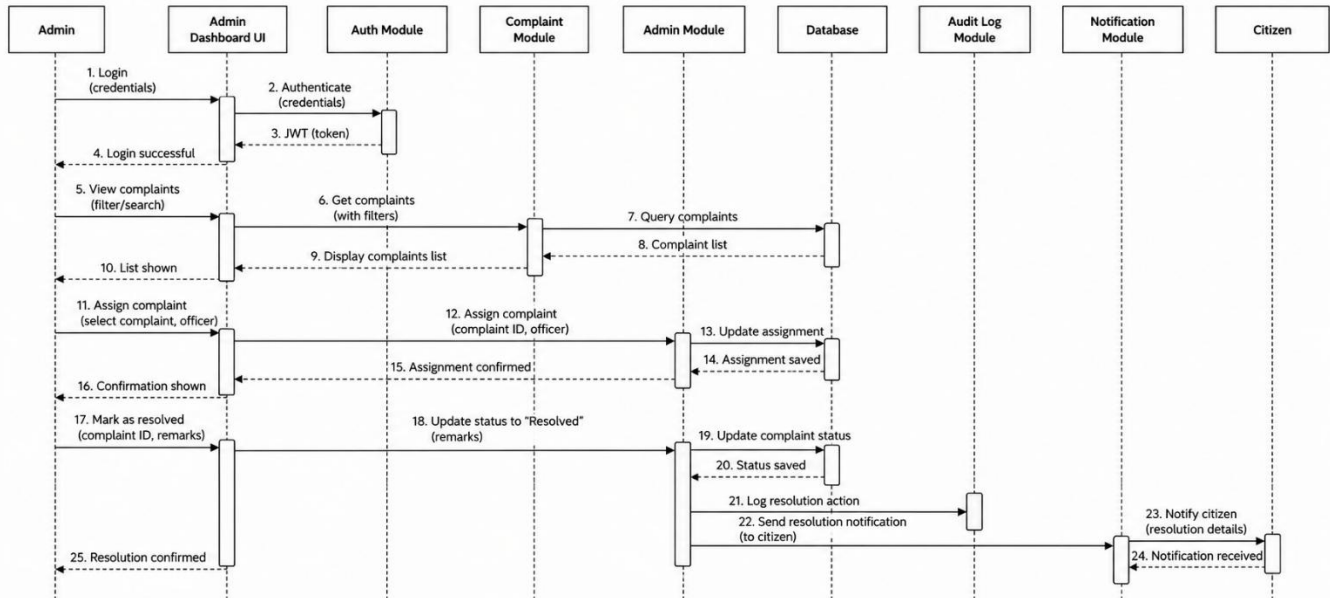
Flow 1 — Citizen lodges a complaint with photo evidence (SCMS-F010–F013, F040)

Flow 1 — Citizen lodges a complaint with photo evidence



Flow 2 — Admin triages, assigns, and resolves a complaint (SCMS-F030-F035, F040)

Flow 2 – Admin triages, assigns, and resolves a complaint (SCMS-F030-F035, F040)



4.3 API Design

Below are two of our main endpoints, shown as an example of how our REST API is designed. All endpoints use JSON over HTTPS (TLS 1.2+) and, unless stated otherwise, need a valid JWT bearer token.

Endpoint: POST /api/v1/complaints

Creates a new complaint for the logged-in citizen (SCMS-F010–F013). Content-Type: multipart/form-data, so we can attach photo evidence.

Request header: Authorization: Bearer <JWT>

```
{
  "category": "Sanitation",
  "description": "Overflowing garbage bin on 5th Cross Road",
  "location": "5th Cross Road, Sector 4",
  "photos": ["evidence1.jpg", "evidence2.jpg"] // up to 3 files, 5MB each
}
```

Success response — 201 Created:

```
{
  "complaintId": "SCMS-2026-00042",
  "status": "Submitted",
  "createdAt": "2026-09-15T09:32:10Z"
}
```

Error responses:

- 400 Bad Request — the request body is malformed
- 401 Unauthorized — the JWT is missing, invalid, or expired
- 413 Payload Too Large — a photo is bigger than 5MB or there are more than 3 photos
- 422 Unprocessable Entity — a mandatory field (category, description, or location) is missing, per SCMS-F012

Endpoint: POST /api/v1/auth/login

Logs in a registered citizen or admin/officer and returns a JWT (SCMS-F002, SR002).

```
{
  "emailOrMobile": "citizen@example.com",
  "password": "*****"
}
```

Success response — 200 OK:

```
{
  "token": "<jwt>",
  "expiresIn": 3600,
  "role": "Complainant"
}
```

Error responses:

- 401 Unauthorized — wrong email/password (we return a generic error message, as required by SCMS-F-002)
- 423 Locked — the account has been locked after 5 failed attempts (SCMS-F004); the user has to wait 15 minutes

4.4 Error Handling, Logging & Monitoring

Every API error is returned in the same format: { "error": { "code", "message", "details" } }. We never log passwords, tokens, or full PII, in line with SCMS-F003 and SCMS-NF003.

Status Code	Meaning	Typical Trigger
401 Unauthorized	Authentication failure	Missing/invalid/expired JWT, or wrong login credentials.
403 Forbidden	RBAC violation	A Citizen account trying to reach an Admin/Officer route (SCMS-SR002).
422 Unprocessable Entity	Validation failure	Missing mandatory complaint fields (SCMS-F012), or malformed input.
500 Internal Server Error	Unhandled server-side error	Unexpected failure; logged with a correlation ID, but no stack trace is sent back to the client.

For monitoring, we plan to track: monthly uptime against our 99.5% target (SCMS-NF002); p90 dashboard/search latency against the 3-second target (SCMS-NF001); how often notifications fail to send; failed-login rate and account-lockouts; and regular checks that the audit log hasn't been tampered with (SCMS-SR004).

4.5 UX Design

The SCMS web UI is responsive from 360px (mobile) up to 1920px (desktop), as required by SCMS-NF004, with proper form labels, keyboard navigation, and support for capturing photos directly from a phone camera when lodging a complaint.

Status badges use colours that meet WCAG AA contrast (at least 4.5:1) across the timeline described in SRS Section 4.3: Submitted (blue), In Review (amber), Assigned (indigo), In Progress (orange), Resolved (green), Closed (grey), Reopened (red).

4.6 Open Issues & Next Steps

- Auto-categorising incoming complaints using AI, to reduce manual triage work.
- Adding a map/GIS view so complaints can be visualised and clustered by location.
- A native mobile app — this is out of scope for now (SRS 1.2), but something we'd like to explore later.
- Push notifications as an extra channel, alongside email/SMS/in-app.
- Multi-language support for the citizen-facing portal.

5. Appendices

5.1 Glossary

SCMS, JWT, RBAC, SLA, ORM, MVC, CSRF, XSS, OTP, RTM, VCS, CI/CD. Complaint ID format: SCMS-YYYY-NNNNN (see SCMS-F011).

5.2 References

- SRS_Smart_Complaint_Management_System_Team_9, v1.0, 05-09-2026
- OWASP Top 10

- NIST SP 800-160
- IEEE 42010 (Systems and software engineering — Architecture description)

5.3 Tools

- PlantUML — for the component and sequence diagrams
- draw.io — for extra diagramming
- Swagger / Postman — for documenting and testing the REST API
- GitHub — version control; Jira — tracking requirements/tasks